实验步骤

怎么用这份文件

每一步分三段：

- **敲：** 要执行的操作
- **该看到：** 对照屏幕。**不一样不要往下走。**
- **要记下：** 报告 6 个问题要用的数字，先记在纸上或单独一个文本文件

每一步开头标 **【本节要填 N 处空】**。四个任务一共 **25 处空**（6 + 8 + 7 + 4）—— 全部填完，整条链才跑得通。

本文里的 `[···]` 表示要你自己跑出来才知道的内容。不要猜，跑一遍就有了。

这份文件只解释每个空**要做什么、输入输出是什么、写错了会怎样**； 具体表达式要你自己写。先读代码里的 `TODO` 注释和函数 docstring， 文件中已经给了类型、公式和库函数上的提示。

卡住了怎么办？ 跳到文末《卡住了怎么办》，按**填空编号**查思路。

> `code/fill.py` 是填空机制，不要改；你只改四个 `task*.py`。 把 `FILL[n]` 这几个字整体替换成你的表达式或语句，不要给它赋值。

第 0 步　环境自检

敲：

```
cd code          # cd 到code目录
python check_env.py
```

该看到：

```
1. 解释器
  ✓ Python 版本 >= 3.8
  ✓ 不是用 -I 启动的
...
填空进度
  task1_md5.py            ▒▒▒▒▒▒   6 / 6
  task2_extend.py         ▒▒▒▒▒▒▒▒  8 / 8
  task3_collision.py      ▒▒▒▒▒▒▒  7 / 7
  task4_defense.py        ▒▒▒▒   4 / 4

  合计 25 / 25

全部通过 —— 可以开始实验了。
```

如果卡住： 不要加 `-I` 或 `-E`。加了会把脚本目录移出 `sys.path`，`from fill import ...` 会失败。去掉参数即可。

第 1 步　任务一：MD5 的填充与续写

【本节要填 6 处空 —— `task1_md5.py` 的 `FILL[1]` ~ `FILL[6]`】

1.1 先看填空长什么样

敲：

```
python task1_md5.py
```

该看到：

```
FILL[1] not filled yet -- task1_md5.py 第 50 行:
    | head = FILL[1]
把 FILL[1] 这几个字删掉，在原位写你的代码。
（退出码 3：还有空没填 —— 这不是程序出错。查进度：python run_all.py --progress）
```

> 退出码 3 是正常状态。它表示「还没填」，不是「代码写错了」。 骨架可以随时 import，只有执行到那一行才会失败。

1.2 填 `FILL[1]` ~ `FILL[3]`：完成 `padding()`

打开 `task1_md5.py`，先读 `padding()` 的 docstring。填充由三部分组成：

```
0x80 | 若干 0x00 | 消息比特数（8 字节，小端序）
```

零的个数要让「0x80 + 零 + 8 字节长度」正好凑到 56 mod 64。

- FILL[1]：填充的**第一个字节**。padding() 返回 bytes，后面要和 零字节串相加。想清楚怎样用 bytes 表示这一个字节， 不要写成字符串或整数。
- FILL[2]：零的**个数**。骨架的 TODO(2) 注释给了公式，代入 msg_len 即可。注意 +1 是在给 0x80 留位置，最后的 8 字节留给长度。 代 55 和 56 两个边界验算一遍。
- FILL[3]：最后的 **8 字节**长度字段。骨架的 TODO(3) 注释提示了打包方式。 注意两点：存的是**比特数**不是字节数，字节序是**小端序**。

> ⚠ 常见错法：FILL[1] 返回 str 或 int，拼接时 TypeError； FILL[2] 少算/多算 1 个零，55 字节消息的填充长度会差 1； FILL[3] 用了大端序或忘了乘 8，前面看起来都正常，续写时才对不上。

1.3 填 FILL[4] ~ FILL[6]：完成 extend_digest()

extend_digest() 要做的是：拿旧摘要当初始状态，继续压 suffix。

- FILL[4]：把 32 个十六进制字符的摘要还原成 **16 字节**的状态。 md5_soft.md5(state=...) 要的是 bytes，不是 str； 直接对字符串做 bytes(...) 不会按十六进制解码。
- FILL[5]：已经处理过的**比特数**。它等于「原消息字节数 + 填充字节数」 再乘 8；md5_soft.md5(count=...) 收的是比特数，不是字节数。别漏了填充。
- FILL[6]：一个**比较表达式**，比较续写摘要和直接哈希整条长消息的摘要。 它必须是真的比较，不能写常量 True。

> ⚠ FILL[6] 写成常量时，程序会打印「两者相同：True」，但两个摘要其实可能不同。 交之前一定看两个十六进制串，不要只看 True。

1.4 跑

敲：

```
python task1_md5.py
```

该看到（数值部分自己跑）：

```
一、填充长什么样
  消息   0 字节 → 填充 [···] 字节，补后 0 mod 64  开头 [···]
  消息   3 字节 → 填充 [···] 字节，补后 0 mod 64  开头 [···]
  消息  55 字节 → 填充 [···] 字节，补后 0 mod 64  开头 [···]
  消息  56 字节 → 填充 [···] 字节，补后 0 mod 64  开头 [···]
  消息  64 字节 → 填充 [···] 字节，补后 0 mod 64  开头 [···]

二、用旧摘要续写一段 suffix
  original = b'Use HMAC, not hashes'
  H(original) = [···]
  填充 = [···]

  从旧状态续写 suffix：[···]
  直接哈希整条长消息：[···]
  两者相同：True

  再验两个边界：
    消息 55 字节：✓
    消息 56 字节：✓
```

> 重点看 55 和 56 两个长度：一个填充很短，一个会多出一整块。 具体是多少，自己跑；报告任务一第 1 题要填。
>
> 报告表还要 20 字节一行；程序没打印的话，用 `padding(20)` 看长度， 或按同样的规则算。

要记下（报告任务一第 1 题用 —— 填充长度表）：

消息长度（字节）	填充长度（字节）	填充后总长 ≡ 0 (mod 64)
0		
3		
20		
55		
56		
64		

要记下（报告任务一第 2 题用 —— 续写记录表）：

项	你的值
原消息长度（字节）	
原消息的填充长度（字节）	
H(original)	

项	你的值
从旧状态续写 suffix 得到的摘要	
直接哈希整条长消息得到的摘要	
两者是否相同	

> 报告任务一第 2 题问「实际只用了哪两样」。看 `extend_digest()` 里 `md5_soft.md5(state=..., count=...)` 的两个关键字参数：它们就是答案。 `original` 只用来算长度，内容没有参与。为什么?
>
> 同一题还问 `FILL[6]` 对不上时先查哪一处。按「零的个数 → count 有没有乘 8 → 有没有算进填充」的顺序写。

如果卡住： 见文末 `task1_md5.py` 的 `FILL[1]~FILL[6]` 六行。

自查思路：

1. 先看 `padding(55)`、`padding(56)` 的长度和十六进制：第一个字节是不是 `0x80`？补后长度是不是 64 的倍数？55 和 56 的填充长度差多少？ 为什么差一整块?
2. 再看 `extend_digest()`：`count` 有没有把填充算进去、有没有乘 8？ `state` 是不是 16 字节?
3. 最后看两个摘要：如果 `FILL[6]` 是 `False`，用 `hashlib` 直接算一遍 长消息作对照，再逐项查填充的 `0x80`、零个数、长度字段。

第 2 步　任务二：长度扩展攻击【★ 主交付物】

【本节要填 8 处空 —— `task2_extend.py` 的 `FILL[1]` ~ `FILL[8]`】

2.1 看输入

敲：

```
Get-Content ../data/sample_url.txt
python api_common.py (Get-Content ../data/sample_url.txt)
```

> 用 cmd 的同学：先 `type ..\data\sample_url.txt` 看内容，再 `set /p URL= <..\data\sample_url.txt`，最后 `python api_common.py "%URL%"`。

该看到：

```
http://bank.cse127.local/api?token=[···]&user=alice&command1=ListSquirrels&command2
status = 200
unlock = False
命令   = {'user': 'alice', 'command1': 'ListSquirrels', 'command2': 'NoOp'}
```

这条 URL 的签名规则是：

```
token = MD5(password || user=...&command1=...&command2=...)
```

密码 8 个字符，只有服务端知道。消息里已经有 &，所以拆 URL 时要注意： 只在**第一个** & 处切。

> api_common.py 是本地模拟的服务端。做任务二时把它当黑盒：只用 token、 消息和密码长度。**不要读源码里的 SECRET**。读了密码再伪造， 就不是长度扩展攻击了。

2.2 填 FILL[1] ~ FILL[3]：拆 URL

打开 task2_extend.py，看 split_url()。

- FILL[1]：query 是 token=...&user=...。要在**第一个** & 处切成两段： 左边是 token=...，右边是完整的消息。注意限制切分次数； 不加限制会把消息里的 & 也切碎。
- FILL[2]：从 token=... 里去掉前缀，留下 32 个十六进制字符。 想清楚切片的起点要跳过几个字符。
- FILL[3]：把消息的百分号编码还原成 bytes，只解码**一次。** 文件顶部已经 import 了对应的解码函数；不要先解码成字符串再编码回来。

> ⚠ 常见错法：FILL[1] 用了不限制次数的切分，消息被切碎； FILL[2] 的 token 里还带着 token=；FILL[3] 解码两次， 消息里出现 %80 字面量。

2.3 填 FILL[4] ~ FILL[8]：伪造

要追加的命令是：

```
&command3=UnlockAllSafes
```

- FILL[4]：算出 password || msg 的 MD5 填充。长度是 **密码长度 + 消息长度**，不是只有消息长度。直接调用你在任务一写的 padding()。
- FILL[5]：新消息 = 原消息 + 填充 + suffix。三段都要有，顺序不能变； 填充里有 0x80 和 0x00，这是服务端重新计算时需要的。
- FILL[6]：新消息已经处理过多少**比特**？要把密码长度、消息长度、 填充长度都算进去，再乘 8。md5_soft.md5(count=...) 收比特数。
- FILL[7]：把 suffix 喂给哈希对象。注意这是**一条语句**，不是表达式； 不要嵌进别的调用里。new_msg 里也要有 suffix，两处都要。
- FILL[8]：拼回完整 URL。保留原来的 scheme、域名和路径， 把 token 换成新 token，把消息部分换成 new_msg 的百分号编码。 用文件顶部已经 import 的字节编码函数，让 &、=、% 都按字节编码； 否则服务端会把填充里的 & 当成参数分隔符。

> ⚠ 最隐蔽的错在 FILL[7]：suffix 拼进了 new_msg 但没喂给哈希对象， 或者反过来。两种都会让 unlock=False。

2.4 跑

敲：

```
python task2_extend.py
```

该看到：

```
原始请求：
    status = 200
    unlock = False
    command3 = （没有）

伪造请求：
    status = 200
    unlock = True
    command3 = UnlockAllSafes
```

★ 成功。服务端认为这条请求是 `alice` 发的。

程序还会打印 token、消息长度、填充长度、新消息长度；这些都是 `[···]`， 自己跑。也可以再单独验证一次写出的 URL：

```
python api_common.py (Get-Content ../data/forged_url.txt)
```

> cmd 写法同 2.1：先 `set /p URL=<..\data\forged_url.txt`，再 `python api_common.py "%URL%"`。

该看到： `status = 200`、`unlock = True`。

要记下（报告任务二第 1 题用 —— 两次请求表）：

项	原始请求	伪造请求
status		
unlock		
token 前 8 位		
请求里的 command		

要记下（报告任务二第 1 题用 —— 长度账）：

项	你的值
密码长度（字节）	8（已知）
原消息长度（字节）	
填充长度（字节）	
新消息长度（字节）	
新消息里 &command3= 之前是什么	

伪造 URL 已由程序写入 data/forged_url.txt，报告任务二第 1 题要 整条贴出来（不要截断、不要换行）。

> 长度扩展攻击最后一行问「新消息里 &command3= 之前是什么」。把 new_msg 用 repr() 打印出来看；填充里有不可见字节，不要只复制可见文本。

> 填完长度账，想一想：四个数里哪一个必须把密码长度算进去才对得上？ 为什么？报告任务二第 1 题要答这一句。
>
> 报告任务二第 2 题不用填表：按「服务端收到什么 → 它怎么算 → 为什么对得上」三段写，再写修复方案，并回答「只把 MD5 换成 SHA-256 够不够」。

如果卡住：

- status=403：先查 FILL[4] 的填充长度有没有加密码长度， 再查 FILL[6] 的 count 有没有乘 8。
- status=200 但 unlock=False：查 suffix 有没有同时进 new_msg 和哈希对象。
- URL 解析后消息长度不对：查 FILL[8] 的编码函数有没有让 &、= 按字节编码。
- unlock=True 但 token 没变：查 FILL[7] 是不是真的喂了 suffix， FILL[4]/FILL[6] 是不是用了旧 token 的状态和错误的 count。

第 3 步　任务三：碰撞与同哈希程序

【本节要填 7 处空 —— task3_collision.py 的 FILL[1] ~ FILL[7]】

3.1 先看两个碰撞块

敲：

```
(Get-FileHash ../data/collision_a.bin -Algorithm MD5).Hash.ToLower()
(Get-FileHash ../data/collision_b.bin -Algorithm MD5).Hash.ToLower()
```

该看到： 两行完全一样的 32 位十六进制。

再看 SHA-256：

```
(Get-FileHash ../data/collision_a.bin -Algorithm SHA256).Hash.ToLower()
(Get-FileHash ../data/collision_b.bin -Algorithm SHA256).Hash.ToLower()
```

该看到： 两行不同。

> 两行具体是什么需要复制，报告任务三第 1 题要用。

3.2 填 `FILL[1]`：找差异

`find_diffs()` 逐字节比较 A 和 B，相同的字节跳过。

- `FILL[1]`：往 `out` 里追加一个**三元组**：偏移 `i`、异或值 `x ^ y`、 异或值里 1 的个数。三个元素的顺序和含义看函数 docstring； 第三项可以数一数这个整数里有几个 1，整数自带对应的方法。
- 不要只追加偏移；报告要的是「几个字节不同、合计几个比特」。

> ⚠ 常见错法：忘了跳过相同的字节，把整份文件都记进去； 或者三元组顺序写反，后面的统计全错。

3.3 填 `FILL[2]` ~ `FILL[3]`：做 zipapp

`build_payload()` 把 `PAYLOAD_CODE` 打成一个 zip。

- `FILL[2]`：往 zip 里写一个名为 `__main__.py` 的文件，内容是 `PAYLOAD_CODE`。`zipfile` 的对象 `z` 和 `ZipInfo` 对象 `info` 都已经准备好了；想清楚用哪个方法写字符串内容。
- `FILL[3]`：返回 zip 的**字节内容**。`buf` 是 `BytesIO`， 直接返回它得到的是对象，不是 `bytes`；要取它里面的值。

> `info` 里的时间戳是固定的，这样每次构建出来的字节完全一样。 不要改这个时间戳，否则 good/evil 的 SHA-256 比较没有意义。

3.4 填 `FILL[4]` ~ `FILL[6]`：接上碰撞块

- `FILL[4]`：good = 碰撞块 **A** + payload。碰撞块必须在前，zipapp 在后； 顺序反了，程序读不到自己的第 `0x13` 字节。
- `FILL[5]`：`evil` = 碰撞块 **B** + payload。两个文件接的是**同一份** payload。
- `FILL[6]`：用当前 Python 解释器运行 `path`，捕获标准输出。 看 `TODO(6)` 里的 `subprocess.run` 提示；需要捕获输出、文本模式， 并设一个超时，避免卡死。返回的对象要有 `.stdout`。

> ⚠ 常见错法：good/evil 用了同一个碰撞块（两份文件完全一样）； payload 各构建一次、时间戳不同；`subprocess.run` 没捕获输出， 访问 `.stdout` 报 `AttributeError`。

3.5 填 `FILL[7]`：验证

- `FILL[7]`：比较两个文件的 MD5，必须是真实比较，不能写 `True`。 两边都用同一种表示：都取字节摘要，或都取十六进制字符串， 不要拿 `bytes` 和 `str` 比。

3.6 跑

敲：

```
python task3_collision.py
```

该看到（具体数值自己跑）：

```
MD5     A = [···]
MD5     B = [···]    相同
SHA-256 A = [···]…
SHA-256 B = [···]…    不同

逐字节比较：[···] 个字节不同，合计 [···] 个比特。
···
加共同前缀：MD5 还相同？ [···]
加共同后缀：MD5 还相同？ [···]

good 输出：[···]
evil 输出：[···]
```

再单独运行两个程序：

```
python ../data/good
python ../data/evil
```

该看到： 两个程序各打印一句不同的话。 报告任务三第 1 题要填。

要记下（报告任务三第 1 题用 —— 碰撞记录表）：

项	你的值
碰撞块长度（字节）	
MD5(collision_a)	
MD5(collision_b)	
两者的 SHA-256 是否相同	
差异字节数 / 差异比特数	
差异偏移（逐字节比较的输出）	
加共同前缀后 MD5 是否仍相同	
加共同后缀后 MD5 是否仍相同	

项	你的值
good 与 evil 的 MD5 是否相同	
good 的输出	
evil 的输出	

> 报告任务三第 1 题还要解释：两个文件 MD5 相同，为什么运行输出不同？ 回到「程序读自己第 `0x13` 字节」。
>
> 后半问「校验和能证明什么、不能证明什么」，用 good/evil 的结果作答。

如果卡住：

- `MD5` 相同：`False`：两个程序是不是接了同一份 payload？ 碰撞块 A/B 有没有弄反？
- `evil` 也打印 good 的话：zipapp 读的位置或碰撞块 A/B 弄反了； 检查 good/`evil` 的前 128 字节。
- 两个文件大小不一样：payload 是不是构建了两次？时间戳固定了吗？
- 前缀/后缀结论反了：回到 Merkle–Damgård 的状态传递。 后缀保留碰撞，前缀破坏碰撞。报告任务三第 1 题要解释为什么。

第 4 步　任务四：HMAC 防御

【本节要填 4 处空 —— `task4_defense.py` 的 `FILL[1]` ~ `FILL[4]`】

4.1 填 `FILL[1]`：朴素构造

`naive_mac()` 要把 `secret` 和 `msg` 直接拼起来，算 SHA-256， 返回十六进制字符串。注意是 SHA-256，不是 MD5；要十六进制形式， 不是原始字节。

> 这个构造就是任务二里 `MD5(password || message)` 的同构写法。 先记住它，下面要和 HMAC 对比。

4.2 填 `FILL[2]`：HMAC

`hmac_mac()` 用标准库 `hmac`，算法用 SHA-256。想清楚 `hmac.new()` 的三个参数顺序：密钥、消息、摘要算法。返回值同样要十六进制字符串。

4.3 填 `FILL[3]`：恒定时间比较

`verify_hmac()` 要重新算一遍 HMAC，再和传入的 `tag` 比较。 不要用 `==`；用标准库里的**恒定时间比较函数**。两个参数都是十六进制 字符串，可以直接比。

> 为什么不用 `==`：普通字符串比较在第一个不同字符处返回， 攻击者可以通过计时逐字节猜 tag。报告任务四第 1 题要解释这一点。

4.4 填 FILL[4]：正确 tag

main() 里已经算好了 mac。FILL[4] 要调用 verify_hmac() 校验**正确**的 mac；参数顺序看函数定义。返回值应当是 True。 下一行的错误 tag 用同一个函数校验，应当是 False。

4.5 跑

敲：

```
python task4_defense.py
```

该看到：

```
同一条消息，两种写法：
  SHA-256(secret || m) = [···]
  HMAC-SHA-256         = [···]

长度扩展演示（用 MD5 做同构演示）：
  H(secret || m)     = [···]
  续写后             = [···]
  直接哈希整条长消息 = [···]
  相同：True

HMAC 校验：
  正确 tag 通过：True
  错误 tag 通过：False
```

要记下（报告任务四第 1 题用 —— 两种 MAC 写法对照表）：

项	你的值
SHA-256(secret ‖ m)	
HMAC-SHA-256(secret, m)	
两种写法结果是否相同	
长度扩展演示：续写摘要与直接摘要是否相同	
正确 tag 是否通过	
改过的 tag 是否通过	

> 报告任务四第 1 题还要写 HMAC 的两层结构，以及 compare_digest 和 == 的区别。前者看原理文档 §6.1，后者看 §6.4。

如果卡住：

- FILL[1] 返回 bytes：检查有没有取十六进制字符串。
- FILL[2] 报 TypeError：检查 hmac.new() 的参数顺序和摘要算法。
- 正确 tag 也不通过：检查 FILL[3] 的重新计算和比较函数； 确保两边都是十六进制字符串，并且没有用 ==。
- ok_good=False：FILL[4] 传的是不是正确的 mac， 不是空串或错误 tag。

第 5 步　收尾清单

敲：

```
python run_all.py
python verify.py
```

该看到：

```
通过 [···] 项，失败 0 项
全部通过。可以交报告了。
```

交什么

文件	说明
code/task1_md5.py ~ task4_defense.py	填完的代码
data/forged_url.txt	伪造的 URL
data/good、data/evil	两个同 MD5 的程序
data/task1_padding.json	任务一的数据
data/defense.json	任务四的数据
实验报告	按模板填写，6 个问题都要作答，「问题和建议」也写

报告「问题和建议」不要空着：写你卡在哪个 FILL[n]、当时什么现象、 最后怎么定位并改对的。没卡住就写你确认过的检查点——「没有问题」也是一种 记录，但要写出你确认过的地方。

data/ 里由你跑出来的文件，随报告一起打包提交。

卡住了怎么办

把程序原样再跑一遍，读它最后几行。 本包的错误信息会给出文件名 + 行号 + 提示。

下面的表按**填空编号**索引。表里只给检查方向，不给代码。

task1_md5.py

编号	现象	先查什么
FILL[1]	拼接时 TypeError，或填充开头不是 0x80	返回的必须是 bytes，不是 str/int；这一个字节的值看 padding() docstring 里的填充结构
FILL[2]	55 字节消息的填充长度差 1，或 55/56 边界不对	公式里的 +1 是给 0x80 留位置；代入后检查 msg_len + 填充长度 是不是 64 的倍数
FILL[3]	前面长度都对，续写却对不上	长度字段是**比特数**、8 字节、**小端序**；检查有没有乘 8、字节序有没有反
FILL[4]	ValueError，或状态长度不是 16 字节	32 个十六进制字符要还原成 16 字节；不要直接对字符串取 bytes()
FILL[5]	续写摘要和直接摘要不同	count 是已处理的**比特数**，要包含填充并乘 8；不要只算原消息
FILL[6]	程序打印「相同：True」但两个摘要不一样	这里必须是真实比较，不能写常量 True；先看两个十六进制串
任意	不要给 FILL[n] 赋值	把 FILL[n] 这几个字整体删掉，在原位写表达式/语句

task2_extend.py

编号	现象	先查什么
FILL[1]	ValueError: not enough values to unpack，或消息被切碎	只在**第一个** & 处切；限制切分次数，右边整段都是消息
FILL[2]	token 里还带着 token=	切片的起点要跳过前缀的长度；数一下 token= 有几个字符
FILL[3]	消息里出现 %80 字面量，或长度不对	百分号解码成 bytes，只解码一次；不要先解码成 str 再编码
FILL[4]	status=403	填充长度用的是**密码长度 + 消息长度**，不是只有消息长度；直接复用任务一的 padding()
FILL[5]	status=200 但 unlock=False	新消息里必须有原消息、填充、suffix 三段，顺序不能变
FILL[6]	token 对不上，或 status=403	count 是比特数：密码长度、消息长度、填充长度都要算进去，再乘 8
FILL[7]	token 没变，或 unlock=False	suffix 要喂给哈希对象，同时也要拼进 new_msg；检查这是语句还是表达式

编号	现象	先查什么
FILL[8]	服务端把填充里的 & 当成参数分隔符	消息部分要用字节编码函数，让 &、=、% 都按字节编码；保留原 URL 的 scheme/域名/路径
任意	status=403 且以上都对	看一下 token 是不是 32 个十六进制字符；状态是不是 16 字节

task3_collision.py

编号	现象	先查什么
FILL[1]	差异列表为空，或每项不是三元组	只记录不同的字节；三个元素是偏移、异或值、1 的个数；检查跳过相同字节的位置
FILL[2]	zip 是空的，或 good 运行报找不到 __main__.py	写入的文件名必须是 __main__.py；内容用 PAYLOAD_CODE；ZipInfo 已经准备好了
FILL[3]	返回 None/BytesIO 对象，或 len(payload) 报错	返回的是 buffer 里的**字节内容**，不是 buffer 对象本身
FILL[4]	good 不是以碰撞块 A 开头，或 evil 打印 good	顺序是「碰撞块 + payload」；good 用 A，evil 用 B；别弄反
FILL[5]	good 和 evil 完全一样	evil 要用碰撞块 **B**，不是 A；两份 payload 必须是同一份
FILL[6]	AttributeError: 'CompletedProcess' has no attribute 'stdout'	subprocess.run 要捕获输出、文本模式；检查参数名，别漏捕获选项
FILL[7]	程序说 MD5 不同，但文件其实一样	两边都用同一种摘要表示；不要拿 bytes 和 str 比；不要写常量 True

task4_defense.py

编号	现象	先查什么
FILL[1]	返回 bytes，或值不是 SHA-256	先拼 secret 再拼 msg，顺序不能反；用 SHA-256；返回十六进制字符串
FILL[2]	TypeError，或 HMAC 值不对	hmac.new() 的参数依次是密钥、消息、摘要算法；最后取十六进制
FILL[3]	正确 tag 也不通过	重新算 HMAC，再和 tag 做恒定时间比较；两边类型要一致
FILL[4]	ok_good=False	传的是正确的 mac，不是空串/错误 tag；函数参数顺序要对

环境和运行

现象	原因	怎么办
`ModuleNotFoundError: No module named 'fill'`	不在 `code/` 目录，或用了 `-I/-E`	先 `cd` 到 `code/`；去掉 `-I`、`-E`
`FILL[n] not filled yet`，结尾退出码 3	那一处空还没填	这是正常状态；打开提示的文件和行号，把 `FILL[n]` 换成你的代码
终端中文乱码	终端不是 UTF-8	用 `python -X utf8 xxx.py`
`python` 打开应用商店	Windows 应用别名	用 `py -3 xxx.py`，或关掉别名
`data/good` 运行报 `SyntaxError`	用文本编辑器改过二进制文件	重新运行 `python task3_collision.py`